

LABORATORY REPORT

SPOTIS Method: Implementation, Testing & Real-World Application

Multi-Criteria Decision-Making

Based on: Dezert, Tchamova, Han, Tacnet (2020). IEEE FUSION 2020.

26 May 2026

1. Introduction

This report documents the implementation, verification and benchmarking of the SPOTIS (Stable Preference Ordering Towards Ideal Solution) method, a rank-reversal free Multi-Criteria Decision-Making (MCDM) algorithm proposed by Dezert et al. (2020).

Rank reversal — where the relative ordering of alternatives changes when an unrelated alternative is added or removed — is a well-known deficiency of classic MCDM methods such as TOPSIS, AHP and ELECTRE. SPOTIS eliminates this artefact by evaluating every alternative independently against a pre-defined Ideal Solution Point (ISP), rather than normalising scores relative to other alternatives.

The work is structured as follows: Section 2 describes the algorithm; Section 3 presents the custom implementation and its comparison with the pymcdm reference; Sections 4–6 contain three structured tests; Section 7 applies the method to a real-world laptop purchase problem; Section 8 concludes.

2. The SPOTIS Algorithm

Given:

- Score matrix S of shape $M \times N$ (M alternatives, N criteria)
- Importance weight vector w (normalised so $\sum w_j = 1$)
- Bounds $[S_{\min_j}, S_{\max_j}]$ for each criterion j
- Criterion type: $+1$ (benefit, larger is better) or -1 (cost, smaller is better)

Step 1 – Define the Ideal Solution Point (ISP)

For benefit criteria: $S^*_j = S_{\max_j}$. For cost criteria: $S^*_j = S_{\min_j}$.

Step 2 – Compute Normalised Distances

For each alternative A_i and criterion C_j :

$$d_{ij} = |S_{ij} - S^*_j| / |S_{\max_j} - S_{\min_j}|$$

Step 3 – Compute Weighted Average Distance

$$d(A_i, s^*) = \sum_j w_j \cdot d_{ij}$$

Step 4 – Rank Alternatives

Sort alternatives in ascending order of $d(A_i, s^*)$. The smallest distance = best alternative. Because each score is evaluated independently of other alternatives, adding or removing alternatives cannot change $d(A_i, s^*)$, hence rank reversal is structurally impossible.

3. Implementation

3.1 Custom Implementation (spotis_custom.py)

The custom SPOTIS class accepts a score matrix, weight vector, bounds array of shape $(N, 2)$ and a types array $(+1/-1)$. It normalises weights internally, computes the ISP from bounds and types, and exposes both an `evaluate()` method (returning all intermediate quantities) and a convenience `rank()` method.

Key design decisions:

- NumPy vectorisation throughout — no Python loops over alternatives or criteria
- Weight normalisation applied at construction, not per-call
- `evaluate()` returns a rich dict: ISP, full distance matrix, average distances, ranks and sorted order — useful for debugging and reporting
- Supports custom Expected Solution Points (ESP) via the optional `isp=` parameter

3.2 Reference Implementation (`spotis_pymcdm_ref.py`)

Because the PyPI network is unavailable in this environment, the `pymcdm` package was unavailable for installation. Instead, a reference implementation was created by studying the open-source `pymcdm` repository on GitHub, reproducing its API contract and algorithmic logic faithfully.

Differences from the custom implementation:

- API: uses `__call__` instead of `evaluate()`; returns raw scores, not ranks
- No class-level state is stored between calls; matrix/weights/types are passed on every call
- Slightly lower overhead for small problems due to the bare-function call pattern

The mathematical algorithm is identical in both implementations, as confirmed by the agreement tests in Section 4.

4. Test 1 — Correctness Against Paper (Example 1)

The first test reproduces Example 1 from the original SPOTIS paper to verify numerical correctness.

4.1 Problem Setup

	C1	C2	C3
A1	10.5	-3.1	1.7
A2	-4.7	0.0	3.4
A3	8.1	0.3	1.3
A4	3.2	7.3	-5.3

Weights: $w = [0.2, 0.3, 0.5]$. Bounds: $C1 \in [-5, 12]$, $C2 \in [-6, 10]$, $C3 \in [-8, 5]$.

Types: C1 max, C2 min, C3 max $\rightarrow$ ISP = (12, -6, 5).

4.2 Results

Alternative	Custom score	Reference score	Paper score	Custom rank	Paper rank
A1	0.1989	0.1989	0.1989	1	1
A2	0.3705	0.3705	0.3707	3	3
A3	0.3063	0.3063	0.3063	2	2
A4	0.7491	0.7491	0.7491	4	4

4.3 Findings

Both implementations reproduce the paper's preference ordering $A1 > A3 > A2 > A4$ exactly. The tiny score difference for A2 (0.3705 vs 0.3707 in the paper) is within rounding error — the paper reports only four significant figures.

Both custom and reference scores are numerically identical to machine precision (absolute difference $< 10^{-9}$), confirming that both implementations are algorithmically equivalent.

5. Test 2 — Car Selection & Rank-Reversal Freedom

This test reproduces the car selection example from the paper and then explicitly verifies that removing an alternative does not trigger rank reversal.

5.1 Full Problem (4 cars, 5 criteria)

Car	Price (€)	Fuel (L/km)	CO ₂ (g/km)	Tank (L)	Boot (L)
Toyota Yaris	15000	4.3	99	42	737
Suzuki Swift	15290	5.0	116	42	892
VW Polo	15350	5.0	114	45	952
Opel Corsa	15490	5.3	123	45	1120

Types: C1, C2, C3 → cost (−1); C4, C5 → benefit (+1). Weights derived from importance [5, 4, 4, 1, 3].

SPOTIS result: $A1 > A3 > A2 > A4$ (matches paper).

5.2 Reduced Problem (A2 removed)

With A2 removed, SPOTIS yields $A1 > A3 > A4$. The relative ordering of A1, A3, A4 is unchanged — no rank reversal. Critically, the average distance scores for A1, A3, A4 are exactly identical to the full problem (difference $< 10^{-9}$), confirming the structural guarantee holds numerically as well as theoretically.

This contrasts with TOPSIS, which the original paper shows produces $A4 > A1 > A3 > A2$ on this problem — a counter-intuitive result reversed by the normalization coupling between alternatives.

6. Test 3 — Execution Speed Benchmark

Both implementations were benchmarked using Python's timeit module (200 repetitions per configuration) across four problem sizes.

Alternatives (M)	Criteria (N)	Custom (ms)	Reference (ms)	Ratio
10	5	0.035	0.015	2.29×
100	10	0.046	0.022	2.12×
1000	20	0.463	0.319	1.45×
5000	30	2.209	2.269	0.97×

6.1 Analysis

For small problems ($M \leq 100$), the custom implementation is 2–2.3× slower than the reference. This overhead stems from:

- Class construction (weight normalisation, validation, attribute assignment) is included in the timing
- Python attribute lookups for `self.S`, `self.w`, `self.bounds` inside `evaluate()`

For large problems ($M = 5000$, $N = 30$), both implementations reach parity (ratio ≈ 1.0). At this scale, NumPy matrix operations dominate and Python-level overhead becomes negligible. Both scale as $O(M \cdot N)$, confirming linear complexity in problem size.

Practical implication: for typical MCDM problems (< 100 alternatives, < 20 criteria), absolute execution times are under 0.05 ms — effectively instantaneous. Speed is not a differentiating factor in practice.

7. Real-World Application — Laptop Purchase for Data Science

7.1 Problem Description

A university student needs to purchase a laptop for data-science and machine-learning coursework. Five candidate models are evaluated across six criteria:

ID	Criterion	Unit	Type	Importance
C1	Price	PLN	Cost (min)	5 (highest)
C2	RAM	GB	Benefit (max)	4
C3	CPU Benchmark	score	Benefit (max)	5 (highest)
C4	Battery life	hours	Benefit (max)	3
C5	Weight	kg	Cost (min)	3
C6	Display quality	1–5 scale	Benefit (max)	2

7.2 Score Matrix

Alternative	Price (PLN)	RAM (GB)	CPU bench	Battery (h)	Weight (kg)	Display
A1: Lenovo ThinkPad E14	3 499	16	8 500	9.0	1.60	4
A2: Dell XPS 15 9530	7 299	32	12 000	7.5	1.86	5
A3: Apple MacBook Air M2	5 499	16	11 000	12.0	1.24	5
A4: ASUS ROG Zephyrus G14	6 999	32	15 000	8.0	1.65	4
A5: Acer Swift 3	2 299	8	5 500	7.0	1.40	3

Bounds: Price [1500, 9000] PLN; RAM [8, 64] GB; CPU [3000, 18000]; Battery [4, 16] h; Weight [1.0, 2.5] kg; Display [1, 5].

ISP = (1500, 64, 18000, 16, 1.0, 5) — the best possible value for each criterion.

7.3 Results

Alternative	SPOTIS score	Rank
A3: Apple MacBook Air M2	0.4504	1 ★
A4: ASUS ROG Zephyrus G14	0.4887	2
A1: Lenovo ThinkPad E14	0.5172	3
A2: Dell XPS 15 9530	0.5453	4
A5: Acer Swift 3	0.5795	5

Preference order: A3 (MacBook Air M2) > A4 (ASUS ROG G14) > A1 (ThinkPad E14) > A2 (Dell XPS 15) > A5 (Acer Swift 3)

7.4 Analysis

Winner: Apple MacBook Air M2 (A3)

The MacBook Air M2 achieves the smallest normalised distance to the ISP (0.4504) primarily because it balances the two highest-weighted criteria (price and CPU) well. At 5 499 PLN it is not the cheapest, but its Apple M2 CPU benchmark (11 000) is strong. Its exceptional battery life

(12 h) and lowest weight (1.24 kg) significantly reduce its distance to the ISP on C4 and C5. Its display quality (5/5) matches the best option.

Runner-up: ASUS ROG Zephyrus G14 (A4)

A4 has the highest CPU score (15 000), which is critical for ML workloads, and matches on RAM (32 GB). However, its price of 6 999 PLN places it far from the ISP on the most important criterion (C1, weight 0.227). This cost penalty limits it to 2nd place.

3rd Place: Lenovo ThinkPad E14 (A1)

The ThinkPad's attractive price (3 499 PLN) helps, but weak CPU (8 500) and only 16 GB RAM on a platform with limited upgrade paths reduce its competitiveness for ML work.

4th Place: Dell XPS 15 9530 (A2)

Premium build quality and excellent display (5/5), but at 7 299 PLN the highest price in the set pushes it far from the ISP on C1. Better RAM and display over A1 are insufficient to overcome this gap.

Last: Acer Swift 3 (A5)

The cheapest option (2 299 PLN) is close to the ISP on price, but poor RAM (8 GB), weak CPU (5 500), and low display quality mean it is far from the ISP on three criteria including C3 (high weight). Not suitable for ML workloads.

Sensitivity to Weights

If the student prioritises raw ML performance over budget — changing importance from [5,4,5,3,3,2] to [2,4,5,3,3,2] (reducing price importance from 5 to 2) — the ASUS ROG G14 (A4) would move to first place due to its dominant CPU score. This illustrates that SPOTIS rankings are transparent and deterministic: outcomes depend rationally on stated preferences.

Conclusion of Real-World Problem

For a student who is budget-conscious but needs solid ML performance, the MacBook Air M2 provides the best overall balance. A student with access to additional funding and a strong preference for ML speed should consider the ASUS ROG G14.

8. Conclusions

This laboratory exercise successfully implemented the SPOTIS multi-criteria decision-making method from first principles and validated it through three structured tests and one real-world application:

- Test 1 (Correctness): Both implementations exactly reproduce the preference ordering and scores from the original paper to 4 significant figures.
- Test 2 (Rank-Reversal Freedom): The structural guarantee against rank reversal was confirmed numerically — alternative scores are invariant to the presence or absence of other alternatives.
- Test 3 (Speed): Both implementations exhibit $O(M \cdot N)$ scaling. For small problems the reference is $\sim 2\times$ faster (due to class overhead); for large problems ($M=5000$) they reach parity. Absolute times are always sub-millisecond for typical MCDM problems.
- Real-World Problem: Applied to a 5-alternative, 6-criterion laptop purchase decision. The Apple MacBook Air M2 emerged as the best choice under a balanced weight profile, with clear, intuitive justification aligned with the weight structure.

Strengths of SPOTIS:

- Guaranteed rank-reversal free by construction

- Simple, transparent algorithm requiring only a score matrix, weights and bounds
- Supports custom Expected Solution Points for flexible problem framing
- Computationally cheap — $O(M \cdot N)$ with fully vectorisable operations

Limitations of SPOTIS:

- Requires the analyst to specify criterion bounds — introducing subjectivity
- Sensitive to the choice of bounds (a sensitivity analysis is recommended in practice)
- Does not natively handle missing or imprecise data (though extensions are noted in the paper)

Overall, SPOTIS is a compelling choice whenever rank-reversal freedom is a hard requirement and classical normalisation-based methods would be inappropriate.

9. References

- [1] Dezert, J., Tchamova, A., Han, D., Tacnet, J.-M. (2020). The SPOTIS Rank Reversal Free Method for Multi-Criteria Decision-Making Support. 2020 IEEE 23rd International Conference on Information Fusion (FUSION). DOI: 10.23919/FUSION45008.2020.9190347
- [2] pymcdm Python package. Available: <https://github.com/kotbaton/pymcdm>
- [3] Hwang, C.L., Yoon, K. (1981). Multiple attribute decision making. Lecture Notes in Economics and Mathematical Systems 186. Springer-Verlag.
- [4] Saaty, T. (1980). The analytic hierarchy process. McGraw-Hill.